Speaking script — WireGuard for Apache NuttX (Community Over Code, Glasgow)

Deck: `coc-glasgow-slides.html` (and the version on the official template, `coc-glasgow-reveal/`), 35 main slides plus 2 appendix slides, English. This file is **made by** `export_en_script.py` from the speaker notes in the deck (press `S` in the reveal edition, `N` in the standalone one). Do not edit this file by hand. Change the notes in the deck and run the script again, or the two will stop matching. The Japanese practice script is made from the same notes by `export_ja_script.py` and follows this script one to one (same slides, same Essential / Optional / Presenter Notes, same times).

How to read this script

Every slide has up to three parts:

- **Say — Essential:** read these lines, top to bottom. They are a complete talk by themselves, with the links between the slides. A slow, clear voice is assumed.
- **Say — Optional:** add them if there is time. If you skip them, the next sentence still works.
- **Presenter Notes:** never spoken. Sources, the steps of the demo, checks, cautions.

Time

125 words per minute for the spoken text, 10 seconds for each agenda slide, and 3:00 for the live demo (talking and typing). Slide changes and short pauses are inside these numbers.

	Words	Time
Essential only (agenda pauses and live demo included)	2783	**24:26**
Essential + Optional	3474	29:58
of which the live demo	255	3:00
Room left in a 30-minute slot, Essential only		5:34
Room left in a 30-minute slot, Essential + Optional		0:02

Part	Slides	Essential	+ Optional
Opening: title, about me, agenda	1–3	1:34	1:59
① Overview and Background	4–9	4:29	5:24
② The Problem & Solution	10–14	3:24	3:49
③ The Development Plan	15–17	1:45	2:17
④ Version 1: Application-Based Implementation	18–23	3:28	4:30
⑤ Version 2: The Kernel Driver	24–28	3:17	4:52
⑥ Live Demo	29–30	3:10	3:35

Part	Slides	Essential	+ Optional
⑦ Summary and Reflections	31–35	3:19	3:32

The time of each slide, shown in its heading, is the Essential time, added up from the start ("you should be about here").

What must not be cut

The most important slide is 28 (Testing: What Ran Where): the claims a reviewer will check. Keep its caveats: FLAT on real boards, KERNEL and PROTECTED on QEMU only, nothing yet on real hardware for KERNEL or PROTECTED. The slides "What Ping Misses" and "Trust the Clock?" are the appendix (A1, A2) after Thank You: not in the times above, opened with the down arrow only if a question needs them.

Keep the story of the first half (slides 2 and 4–14: About Me, then Overview to Architecture): why I started, Sony and NuttX, my own experience, the problem, why a VPN on the device, why WireGuard. Everything after depends on it. Keep slide 33 (What I learned) too: it is the speaker's own message.

If time is short, cut in this order:

1. Every Optional line.
2. Slide 23 (the Version 1 video still). The live demo shows the same kind of thing.
3. Slide 22 (Bug ③, the 50 ms polling). Its one-line lesson is already in the bug table.

If the live demo stops, say so and go on to the summary (there is no backup video at the moment). **The agenda slides (4, 10, 15, 18, 24, 29, 31) are a ten-second pause each**: point at the highlighted line and say the one sentence. If you have no time at all, skip them.

---

1 — Title (0:00 → 0:27)

On slide: "WireGuard for Apache NuttX", large, with the subtitle "Bringing Secure Remote Access to Embedded Devices". Community Over Code Glasgow 2026 on top; Satoru Akita, Sony Semiconductor Solutions below. A QR code of the slides ("Slides") at the bottom right. No technical details on this slide.

Say — Essential:

- Hello everyone. I'm Satoru Akita from Sony Semiconductor Solutions in Japan.
- Today, I'll share my experience bringing WireGuard, a lightweight VPN, to Apache NuttX.
- The goal was simple: secure remote access to embedded devices.
- I'll talk about why I started this project, how I built it, the challenges I faced, and what I learned along the way.

2 — About Me (0:27 → 1:10)

On slide: Three bands: photo, name line and the fox icon / a map of Japan (Nagoya, Sendai, Kawasaki) labelled Japan / five large photos under "My background & activities": Rocket / Aerospace Engineering,

DNA Nanorobotics, Semiconductor Research, Robots, Open-Source Conference Volunteer.

Say — Essential:

- First, a little about me.
- I am an edge AI engineer at Sony Semiconductor Solutions. I use Apache NuttX from the application side. I was born in Nagoya, I studied in Sendai, and now I live in Kawasaki.
- My background is wide. At university, I studied mechanical and aerospace engineering, including rockets. My undergraduate research was DNA nanorobotics, and my master's research was on semiconductors. I also build robots, and I volunteer at open-source conferences.
- This project is the latest stop on that road. Let me show you the project.

Say — Optional:

- The conferences are Open Source Summit Japan and KubeCon. I also build robots at hackathons.
- My road to this project went through molecular robots, computer vision, edge AI cameras, and embedded systems on SPRESENSE.
- On social media, I am the fox in this picture. I also wrote a book about open source.

3 — Agenda (1:10 → 1:34)

On slide: The seven parts, each a short title with a one-line preview and its time: overview & background, project origins, NuttX & Sony, my experience (~5 min) / the problem & solution, the remote access challenge, why WireGuard (~4) / the development plan, reuse wireguard-lwip, adapt to NuttX (~2) / version 1, application-based implementation, the first working tunnel, debugging challenges (~4) / version 2, the kernel driver, why a driver, integration challenges, validation (~5) / live demo, secure remote access, Stack-chan control (~3) / summary & reflections, achievements, personal reflections, community (~4).

Say — Essential:

- Here is the plan. There are seven parts.
- First, the background: why I started. Then the problem and solution, and the development plan. After that, Version 1 and Version 2. Then a live demo, and a short summary.
- I will finish in about thirty minutes, and then I will take questions.

4 — Agenda: ① Overview and Background (1:34 → 1:44)

Say (one line): Let's start with the background.

Pause: point at the highlighted line, one beat.

5 — Project Overview (1:44 → 2:16)

On slide: The wg0 netdev picture (unchanged): application / NSH, the NuttX network stack, wg0 next to eth0 / wlan0, a UDP socket underneath, the Internet, the peer. Below it three cards: lightweight VPN / virtual interface (wg0) / secure communication over UDP.

Say — Essential:

- Here is the whole project in one picture.
- I brought WireGuard, a lightweight VPN, to Apache NuttX. It appears as a virtual network interface called wg0, next to the normal network interface. Packets sent to wg0 are encrypted. They travel over a UDP socket to any normal WireGuard peer. So a NuttX microcontroller can exchange IP packets securely.
- First, let me tell you how this project started.

6 — How It Started: February to October (2:16 → 3:16)

On slide: A month axis Feb → Oct. Above it: mid Feb, read the GSoC idea list / early Mar, proposed the plan / Mar 20, heard about this conference from Alan, submitted a CfP / early Apr, GSoC proposal / early May, CfP accepted, GSoC not selected / Oct, Glasgow. Below it, three arrows: May–Jul study and exploration → Aug Version 1 (application, FLAT, hardware testing) → Sep– Version 2 (kernel driver).

Say — Essential:

- Here is the timeline.
- In mid-February, I read the GSoC idea list and started to study. In March, I proposed my plan. On March 20, Alan, from the NuttX community, told me about this conference, so I submitted a CfP, a call for proposals. In early April, I sent my GSoC proposal. These were two separate applications.
- In early May, I got two answers. The CfP was accepted, but my GSoC proposal was not selected. I decided to keep going anyway.
- From May to July, I studied NuttX and WireGuard. In August, I focused on hardware testing and finished Version 1. Then I built Version 2, the kernel driver.
- So this work started with GSoC, and it grew step by step. First, what is NuttX?

7 — A Quick Overview of Apache NuttX (3:16 → 4:03)

On slide: Left: logo, four points (POSIX-like real-time OS / small footprint / broad hardware support, 15+ architectures, 300+ boards / Apache open-source project) and a small table of standard APIs (FreeRTOS, Zephyr, NuttX). Right: photo tiles of real devices: Sony, Fitbit Charge 3 / 4, PX4 flight controllers, the SORA-Q lunar robot, and Xiaomi OpenVela (1,000+ products).

Say — Essential:

- NuttX is a real-time operating system for small devices. It follows POSIX, so it feels familiar to Unix programmers. It supports many CPU architectures and boards, and it is an Apache open-source project.
- FreeRTOS and Zephyr are good RTOSes too. For this project, the POSIX-like API matters. With pthreads, sockets and files, a VPN can become a normal network device.
- NuttX is also used in real commercial products. You can see some examples on the screen: audio products, fitness trackers, drone controllers, and even a lunar robot.
- Sony has a long history with NuttX. Let me show it.

Say — Optional:

- Sony uses NuttX in audio products and SPRESENSE. Fitbit used it in fitness trackers, such as the Charge 3 and Charge 4. PX4 drone flight controllers run on it. Xiaomi's OpenVela is reported to run on more than a thousand products. And a small lunar robot, SORA-Q, used it on Japan's 2024 Moon mission.

Presenter Notes:

- **Check before the talk:** the Fitbit models come from Fitbit's own 2020 NuttX talk, and the Xiaomi number is a reported figure.

8 — Sony's History with NuttX (4:03 → 4:52)

On slide: Timeline: 2013–15 early adoption in audio products / 2016–17 commercial products / 2018 Xperia Ear Duo and SPRESENSE / 2019–20 upstream collaboration / 2020s expanded edge-sensing capabilities.

Say — Essential:

- Sony has used NuttX for more than ten years. The public record starts in 2013, when Sony engineers began porting NuttX to an audio processor. The first commercial audio products shipped in 2015.
- In 2018, the Xperia Ear Duo shipped with a Sony chip and NuttX. SPRESENSE also launched, as a development board with GNSS, audio and camera support.
- Later, Sony engineers moved much of their platform and driver work upstream, and shared it in public NuttX talks.
- So Sony's NuttX story began with audio products. It grew through SPRESENSE, and it became long-term open-source collaboration.
- That is Sony's history. Now, my own.

Say — Optional:

- Public product examples include the Walkman NW-WS620, the SMR-10 sound amplifier, and two voice recorders, the ICD-TX800 and the ICD-PX470.
- In the 2020s, the platform's abilities kept growing in camera, GNSS, connectivity and edge sensing. That is about technical capability. It is not a claim that every feature shipped in a named product.

Presenter Notes:

- Sources (not spoken). M. Ishikawa and K. Okamoto, SMP and Networking support on NuttX / LC823450, 2018: https://events.nuttx.apache.org/wp-content/uploads/2019/11/MIshikawa_nx2019.pdf . Sony, Running NuttX on Spresense, NuttX Workshop 2019: https://events.nuttx.apache.org/wp-content/uploads/2019/11/TKoizumi_nx2019.pdf . A. Jerpelea, A Journey from Fork to Mainline, NuttX Online Workshop 2020: https://events.nuttx.apache.org/index.php/past-events/nuttx-online-workshop-2020/ . Apache NuttX, Sony CXD56xx documentation: https://nuttx.apache.org/docs/latest/platforms/arm/cxd56xx/index.html . Product images: Sony NW-WS620 https://www.sony.jp/products/picture/NW-WS620.jpg , Xperia Ear Duo

https://www.sony.jp/xperia/product/smartproducts/xea20/img/main01.jpg , SPRESENSE https://developer.spresense.sony-semicon.com/

9 — My Experience with NuttX (4:52 → 6:03)

On slide: Two photo cards: SPRESENSE, a satellite project (picture of the ARICA CubeSat) and AITRIOS, an edge AI sensing platform (edge AI camera). Bottom line: I wanted a secure way to access and troubleshoot remote embedded devices.

Say — Essential:

- Now, my own experience with NuttX. I have two stories.
- First, SPRESENSE. It is Sony's low-power microcontroller board, and it runs NuttX. The ARICA-2 CubeSat, from Aoyama Gakuin University, uses SPRESENSE as its on-board computer. It sends rapid alerts for gamma-ray bursts, and it was launched in April 2026. I took part in this project through an internal, grassroots engineering initiative.
- Second, AITRIOS, Sony's edge AI sensing platform. It is used to manage edge AI cameras and run inference at the edge. I worked on setting up and operating edge AI camera systems. In the systems I worked with, the board was an ESP32 running Apache NuttX.
- A satellite in orbit and a camera on site are both far from my desk. These experiences made me want a secure way to access and troubleshoot remote embedded devices. That is the problem I want to talk about next.

Say — Optional:

- Not every AITRIOS camera is built this way.

Presenter Notes:

- **Check before the talk:** the ARICA picture is a third-party image. Get the project's permission, or replace it.

10 — Agenda: ② The Problem & Solution (6:03 → 6:13)

Say (one line): Now, the problem and the solution.

Pause: point at the highlighted line, one beat.

11 — Problem: How Can We Securely Access Remote Devices? (6:13 → 7:01)

On slide: Engineer → Internet → router → Wi-Fi → NuttX device, with a bracket for the secure path needed all the way, and WPA2/WPA3 covering only the Wi-Fi hop; three cards: devices are far away / remote diagnosis / a secure path.

Say — Essential:

- Now, the problem I wanted to solve.

- Embedded devices are often installed far away: at a customer's site, in a factory, or on another network. When something goes wrong, an engineer wants to log in, check the device, and fix it. Nobody wants to travel there.
- Wi-Fi security protects the radio link to the access point. That is important, but it is only one hop. It does not give an engineer a way to reach the device across the Internet. For that, we need a secure path of its own.
- So the question is: how can we securely access remote devices?

Say — Optional:

- Wi-Fi security here means WPA2 or WPA3. The same need appears in edge AI, industrial IoT, and many other remote systems.

12 — Options: Ways to Reach a Remote Device (7:01 → 7:51)

On slide: A table of six ways (open a port / HTTPS or MQTT via a cloud / reverse tunnel or relay / VPN on the router / carrier closed network / VPN on the device) against: any IP service, no server in the middle, protected up to where.

Say — Essential:

- There are several ways to do this: open a port, use a cloud API or an MQTT broker, set up a reverse tunnel, or use a VPN on the router.
- Each way has trade-offs. HTTPS and MQTT are great for data, but they do not give you a shell or a web page on the device. A router VPN works well, but its protection ends at the router.
- All of these can be made secure. They just protect different parts of the path.
- For this project, I wanted a standard VPN on the device itself. Then the device is the end of the tunnel.

Presenter Notes:

- **If asked about Tailscale or ZeroTier:** they are also a VPN on the device, but they usually need a bigger OS than a microcontroller has today (footnote on the slide).

13 — Solution: A VPN on the Device (7:51 → 8:51)

On slide: Engineer / Linux / Windows ↔ WireGuard tunnel ↔ NuttX device. "Why WireGuard?": small & simple (a small bar chart of lines of code: IPsec ~420k, OpenVPN ~120k, WireGuard ~3.8k, from 2018) / modern cryptography (ChaCha20-Poly1305, Curve25519) / UDP-based tunnel.

Say — Essential:

- So my solution is a VPN on the device itself. Why WireGuard?
- First, it is small and simple. In a 2018 talk, its author compared the size of the code: about four thousand lines for WireGuard, about one hundred and twenty thousand for OpenVPN, and over four hundred thousand for IPsec. These are old numbers for specific implementations, but the gap is large. That matters on a microcontroller.

- Second, it uses modern cryptography. Third, it is a tunnel over plain UDP. It works with existing WireGuard peers, such as Linux or Windows. And the microcontroller itself becomes the end of the tunnel.
- So the idea is simple: run a lightweight, standard VPN directly on the NuttX device. Now, let me show where it fits in NuttX.

Say — Optional:

- The cryptography is ChaCha20-Poly1305 and Curve25519. I will not go into the details.

Presenter Notes:

- The line counts come from the WireGuard author's Linux Plumbers 2018 talk (slide 5, https://www.wireguard.com/talks/lpc2018-wireguard-slides.pdf). They are old, for specific implementations, and slide 4 of the same talk gives slightly different OpenVPN numbers.

14 — Architecture: WireGuard on the NuttX Network Stack (8:51 → 9:27)

On slide: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0 / usrsock → network, with TX and RX arrows.

Say — Essential:

- Here is where wg0 sits. When an application sends data, it goes down through TCP, UDP and IP, and reaches wg0 like any other network device. WireGuard encrypts it, and it leaves as a UDP packet. Incoming packets take the same path in the other direction.
- From here, my work is building this middle part, without changing the layers above and below it.
- So, how did I bring it to NuttX? That is the development plan.

Say — Optional:

- On a board, the packet leaves through wlan0, or through usrsock when the Wi-Fi chip works through sockets.

15 — Agenda: ③ The Development Plan (9:27 → 9:37)

Say (one line): Next, the development plan. How can we reuse an existing WireGuard implementation?

Pause: point at the highlighted line, one beat.

16 — Porting Strategy: Start from wireguard-lwip (9:37 → 10:33)

On slide: The link github.com/smartalock/wireguard-lwip (clickable), then three rows: KEEP protocol & crypto (with two small fixes) / ADAPT platform hooks / REPLACE lwIP network glue.

Say — Essential:

- I did not write WireGuard from scratch.

- I started from wireguard-lwip, an open-source WireGuard implementation for the lwIP network stack. The link is on the slide. It already has the protocol and the cryptography, so I could focus on the porting.
- I sorted the code into three parts. First, KEEP: the protocol and the crypto. I reused them. The crypto is unchanged, and the protocol core needed two small fixes. Second, ADAPT: the platform hooks, a few functions that depend on the operating system, such as the clock and random numbers. Third, REPLACE: the lwIP network glue. NuttX does not have lwIP, so this layer is the real porting work.
- So, how does lwIP map to NuttX?

Say — Optional:

- The four hooks are a monotonic clock, random bytes, the current time in TAI64N format, and a check for system load. I found the two protocol fixes in a review.

Presenter Notes:

- **Check before the talk:** the four platform hooks are a monotonic clock, random bytes, TAI64N time and an under-load check (`wireguard-platform.h`). Say them from the notes, not the slide.

17 — Mapping lwIP Networking to NuttX (10:33 → 11:12)

On slide: Table: network device (netif → net_driver_s) / packet buffers (pbuf → iob) / registration (netif_add() → netdev_register()) / sending (output callback → d_txavail → devif_poll()) / UDP wire (udp_pcb → UDP socket). A note: matches of ideas, not a one-to-one swap of APIs.

Say — Essential:

- This was the most important porting decision. I did not bring lwIP into NuttX. I used NuttX's own building blocks.
- lwIP's network interface becomes a NuttX net_driver_s. lwIP's packet buffer is similar to NuttX's I/O buffer. Adding an interface becomes registering a device, and I register wg0 as a tunnel-type device. The wire under wg0 is a plain UDP socket.
- These APIs are not exactly the same, but they serve similar purposes. With this design, I could build the first version.

Say — Optional:

- In lwIP, the interface is netif and the buffer is pbuf. In NuttX, the buffer is iob. For sending, lwIP calls an output callback. NuttX calls the driver's txavail function, and the driver polls the stack with devif_poll.

18 — Agenda: ④ Version 1: Application-Based Implementation (11:12 → 11:22)

Say (one line): Now, Version 1: the first working tunnel.

Pause: point at the highlighted line, one beat.

19 — Version 1: WireGuard as a NuttX Application (11:22 → 12:10)

On slide: A GOAL box (run a standard WireGuard tunnel as a NuttX application; reuse the existing core and implement the NuttX integration). A table of layer / work needed / in this port: protocol & crypto (almost none; crypto byte-identical, two small fixes; ~3,100 lines reused, ~40 changed) / OS hooks (four functions; ~120 new lines) / network glue (the real work: netdev + UDP socket + RX task; ~1,200 new lines). Takeaway in two lines: Version 1 proved WireGuard could run on NuttX; it also exposed OS and networking problems the first tunnel did not reveal.

Say — Essential:

- Version 1 is WireGuard as a NuttX application, in a FLAT build. The goal was a standard tunnel that I can configure at run time, without rebuilding the firmware.
- This table shows the work in each layer. The protocol and crypto came from wireguard-lwip, with only two small fixes. The OS hooks are small. The real work was the network glue: the netdev, the UDP socket, and a receive task. That was about twelve hundred new lines.
- It worked, but it also showed problems that a first successful ping could not show. Let me show where it was hard.

Say — Optional:

- About three thousand lines came over from wireguard-lwip. The OS hooks are about one hundred and twenty lines.
- One of the two small fixes was a bug in wireguard-lwip itself: its rate limit never worked.

Presenter Notes:

- Line counts are physical lines from git history. Reused: the vendored wireguard-lwip files (about 3,100 lines). Fixes: about 40 lines changed (2026-09-18). OS hooks: nuttx-platform.c, 118 lines at the first working tunnel (2026-08-13). Glue: nuttx-wireguardif.c and .h, about 1,200 lines at that time. Today's file is about twice as long because of run-time configuration.

20 — Where It Was Hard: Six Bugs (12:10 → 12:45)

On slide: One table of all six bugs (bug / showed up in / lesson): 1 the receive thread stopped with the wg command / 2 ping works, TCP dies (in detail) / 3 "no real impact" was 10× (in detail) / 4 stack 94% full, saving the config froze the board, real boards only / 5 NuttX's own ChaCha20-Poly1305 (part 5) / 6 a 6 KB array on a 3 KB kernel stack (part 5). Takeaway: most hid behind a test that passed.

Say — Essential:

- The first tunnel worked quickly. Most of my time went into six bugs: four in the application version, and two more in the kernel version.
- Most of them hid behind a test that passed, like a handshake or a ping. So at first, I looked in the wrong place.
- I will explain two of them, because their lessons are useful for anyone. Bugs five and six come later, in the kernel part.

Say — Optional:

- Bug one: the receive thread stopped when the wg command that started it exited. Detach only decides when memory is freed. It does not decide how long the thread lives. That is why the kernel driver owns its thread.
- Bug four: only real hardware showed it. On the board, the receive task's stack was ninety-four percent full. The simulator has big stacks, so it never showed. Measure on real hardware.

Presenter Notes:

- The logs and fixes for all six bugs are in the GitHub repository.
- **Bugs 1 and 4 have no slide of their own any more.** Their story is in the notes above; the full version (and bug 1's `SO_RCVTIMEO` detour) is in the Q&A file.

21 — Bug ②: Ping Works. TCP Dies. (12:45 → 13:31) *(my favorite)*

On slide: Orange-red title. Left box SYMPTOM: ping 0% loss; the TCP handshake completes, then the connection dies; EBADF, "Bad file descriptor". Right box ROOT CAUSE & FIX: sends from wg_rx worked, sends from the system work queue failed; a file descriptor belongs to one task group; use psock_*() on a struct socket.

Say — Essential:

- This is my favorite bug.
- On the same tunnel, ping worked perfectly, and even the TCP handshake finished. But then the TCP connection died. The log showed E-B-A-D-F: bad file descriptor.
- Sending worked from the receive task, but failed from another task group, the system work queue.
- In NuttX, a file descriptor belongs to the task group that opened it. It was not broken. It was just not visible from another task group.
- The fix: I used NuttX's internal socket API, on a struct socket. It is just memory, so any task can use it.

Say — Optional:

- That is also why the kernel version keeps its socket as a struct socket.

22 — Bug ③: "No Real Impact" Was 10× Slower (13:31 → 14:16) *(cut candidate 3)*

On slide: Orange-red title. Two boxes: QUICK FIX, check every 50 ms (measured: about 10× slower) / REAL FIX, event-driven wake-up (psock_poll() callback and a semaphore).

Say — Essential:

- The next bug is about something I never measured.
- The fix without file descriptors also broke poll. So, as a quick fix, I checked the socket every fifty milliseconds. I thought the only cost was waking up the CPU. I was wrong. Every packet could wait up to fifty milliseconds, and that made it about ten times slower.

- The real fix was a proper wait: the socket calls back, and a semaphore wakes the thread when data arrives.
- The lesson: a quick fix that you never measure can look free. So, measure it.

Say — Optional:

- Poll broke because poll works with file descriptors.

23 — Version 1 Demo: The First Video (14:16 → 14:40) *(cut candidate 2)*

On slide: Left: the video as a still frame (the Windows WireGuard client beside a telnet session into NuttX on the ESP32-S3) with a play button; a click plays the YouTube video in the slide, and leaving the slide stops it. Right: the steps telnet 10.10.0.2 / uname -a, ifconfig / webserver & / browser http://10.10.0.2/; ESP32-S3 DevKit, apps version (FLAT), USB unplugged; the official Windows client; small text youtu.be/1kyX2av5WG4.

Say — Essential:

- Here is a frame from the Version 1 video. I logged in with telnet through the tunnel, and opened a small web page on the board. The other end was the official Windows WireGuard client.
- So Version 1 was enough for a demo. Next, I made it a real NuttX driver.

Presenter Notes:

- Playing the video is not needed: the Essential text works with the still. To play it, click the thumbnail. It needs the Internet and a web page (GitHub Pages or a local server); opened from a file, the click opens YouTube in a new tab. Leaving the slide stops the video.
- **It is the apps version.** Do not mix it with the kernel-version results later in the talk.

24 — Agenda: ⑤ Version 2: The Kernel Driver (14:40 → 14:50)

Say (one line): Now, Version 2: the kernel driver.

Pause: point at the highlighted line, one beat.

25 — Version 2: WireGuard as a NuttX Kernel Driver (14:50 → 15:46)

On slide: Two panels, BEFORE — VERSION 1 (an application: in apps/, uses kernel internals, FLAT only, does not fit as an in-tree driver) / AFTER — VERSION 2 (a kernel driver: drivers/net/wireguard, set up by a small wg command through ioctl, made for FLAT, PROTECTED and KERNEL). A BUILD MODES box with three rows: FLAT (one address space) / PROTECTED (MPU) / KERNEL (MMU). The goal line.

Say — Essential:

- Why did I move it into the kernel?
- Version 1 is an application. It reaches into kernel internals. That works in a FLAT build, where applications and the kernel share one address space. But it does not fit as an in-tree driver, and it cannot work in PROTECTED or KERNEL builds, where they are separated.

- So I moved the device into the kernel, next to the other network drivers. A small wg command sets it up through ioctl.
- The goal is a proper kernel driver for every build mode, in a form that better fits the NuttX architecture and upstream review. That does not mean it will be accepted. It means it follows how NuttX is built.

Say — Optional:

- In a PROTECTED build, the MPU separates memory. In a KERNEL build, the MMU gives each side its own address space.

26 — The Kernel Design: The Big Picture (15:46 → 16:24)

On slide: A picture in two bands: USER SPACE holds only the wg command; an ioctl (one flat struct, no pointers; the private key is write-only) crosses the user/kernel boundary into the NUTTX KERNEL band, to wg0, which owns its UDP socket and RX thread, keeps timers / peers / cookies and uses NuttX crypto, between the NuttX network stack and the encrypted UDP to the peer.

Say — Essential:

- This is the shape of the driver. The top is user space. The bottom is the NuttX kernel.
- In user space, there is only a small wg command. It sends settings to the kernel through ioctl, as one flat structure with no pointers. The private key is write-only.
- In the kernel, wg0 is a normal network device. It owns its UDP socket and its receive thread. So the thread and socket bugs from Version 1 cannot happen by design.

Say — Optional:

- The crypto is NuttX's own crypto. The protocol core is still derived from wireguard-lwip, under the BSD-3-Clause license, and the LICENSE file says so.

27 — Two Issues Found During Kernel Integration (16:24 → 17:24)

On slide: Two labelled boxes. CRYPTOGRAPHY — ChaCha20-Poly1305 (NuttX crypto/chachapoly.c: the nonce counter sits in different bytes from the RFC 8439 / WireGuard layout; the handshake works, data after the first packet fails; a two-line fix with test vectors) / KERNEL STACK — Stack Overflow (this driver, KERNEL build only: a 6 KB array on a 3 KB kernel stack, moved to the heap). The SPRESENSE RTC_HIRES finding is in the notes only.

Say — Essential:

- On the way into the kernel, I found two separate problems.
- The first is in NuttX's ChaCha20-Poly1305 code. It places the nonce counter in different bytes from the layout in RFC 8439, which WireGuard uses. When the counter is zero, the bytes are the same. So the handshake works, but the data packets after the first one fail. The fix is two lines. I prepared it as a separate change, with test vectors.

- The second is a bug in my own driver, not in NuttX. Setting the peers put a six-kilobyte array on a three-kilobyte kernel stack. That caused a panic, only in a KERNEL build. I moved the array to the heap.
- I prepared these as small, separate pieces. Nothing is submitted yet.

Say — Optional:

- As far as I can see, current NuttX master still has the same nonce layout.
- One more finding from the board side: SPRESENSE did not boot with the high-resolution RTC option, CONFIG_RTC_HIRES, on NuttX 13.0.1 and master. I found the cause and fixed it locally. The report is drafted, but not filed.

Presenter Notes:

- Status on 2026-10-13: no pull request exists in apache/nuttx or apache/nuttx-apps; the only related issue is the closed GSoC thread #18548. The nonce check against master was made on 2026-10-13 (crypto/chachapoly.c).
- **Say it exactly:** the stack overflow was in this driver, not in NuttX. Nothing is filed or sent yet; do not say it is.

28 — Testing: What Ran Where (17:24 → 17:57) *(core — do not cut)*

On slide: A table, build mode / environment / result: FLAT on ESP32-S3 and SPRESENSE (real boards) verified / KERNEL on QEMU rv-virt verified (tunnel to Linux, 4 CPUs under load) / PROTECTED on QEMU rv-virt verified (one fault found and fixed, 20/20) / KERNEL and PROTECTED on real hardware not tested yet. A small note that the check mark means it passed in the conditions noted. The GitHub line and QR code are gone.

Say — Essential:

- Let me be exact about what I tested, and where.
- In a FLAT build, I tested on real boards: the ESP32-S3 and SPRESENSE. In a KERNEL build, I tested on QEMU. In a PROTECTED build, I also tested on QEMU. I have not yet tested KERNEL or PROTECTED on real hardware.
- So a check mark means: it passed under these conditions. It is not a promise for every board.

Say — Optional:

- On October 12, I ran the exact series I prepared for upstream on the ESP32-S3: a handshake with Linux WireGuard, and ping both ways, up to the full packet size.
- On SPRESENSE, I measured memory. After forty down-and-up cycles, it was exactly the same.
- On QEMU with four CPUs, I ran heavy traffic while changing settings. There was no fault. But that was one scripted run, not a full search for race conditions.
- In PROTECTED, repeated runs sometimes faulted. The cause was a stack overflow in the wg command. A bigger default fixed it: twenty out of twenty runs.

Presenter Notes:

- Details. SPRESENSE receive thread used 1,472 bytes of a 6 KB default. PROTECTED stack overflow: a system call runs on the caller's stack in this mode. CoreS3 cannot boot bootloader-based images, so PROTECTED on that board is not possible (not a WireGuard problem). Records are in docs/upstream/evidence.

29 — Agenda: ⑥ Live Demo (17:57 → 18:07)

Say (one line): Time for the live demo.

Pause: point at the highlighted line, one beat.

30 — Live Demo: Stack-chan over WireGuard (18:07 → 21:07)

On slide: One slide, kept on screen with the terminal beside it. Top, NETWORK SETUP: Laptop (WireGuard peer, VPN IP 10.10.0.1) and Stack-chan (Apache NuttX + WireGuard, VPN IP 10.10.0.2), both on a Smartphone (Wi-Fi hotspot, provides the local network) over dashed Wi-Fi links, and one thick red Encrypted WireGuard Tunnel between laptop and Stack-chan. Bottom, LIVE DEMO STEPS: 1 VPN Connection (wg show) / 2 Remote Login (telnet 10.10.0.2) / 3 Network Interface (ifconfig → wg0) / 4 Control Stack-chan (change face, play voice). A small Stack-chan photo that links to its GitHub repository. No Version label, no takeaway box, no video link.

Say — Essential:

- I keep this slide on the screen, and the terminal is next to it.
- My laptop and the Stack-chan are both on my smartphone's Wi-Fi hotspot. The phone only provides the local network. Between the laptop and the Stack-chan, there is one WireGuard tunnel. The laptop is ten dot ten dot zero dot one, and the Stack-chan is ten dot ten dot zero dot two. The phone is not a WireGuard peer. It only carries encrypted packets.
- Step one: the VPN connection. I run wg show on the laptop. The handshake is done, so the tunnel is up.
- Step two: remote login. I use telnet to connect to the Stack-chan through the tunnel. Now we are inside the NuttX shell.
- Step three: the network interface. I run ifconfig. You can see wlan0 on the phone's network, and wg0 with the VPN address.
- Step four: control. First, I change the face. Now I make it speak.
- So the Stack-chan is a NuttX device, and I am controlling it remotely through WireGuard. One note: this robot runs the application version, in a FLAT build. The kernel driver ran on the same chip in a separate test.
- That is the demo. Now, let me summarize.

Say — Optional:

- If there is time, I can show the packets. Outside the tunnel, the phone's network sees only encrypted UDP. Inside the tunnel, you can see the telnet text. Then wg show again: the byte counts went up.
- The voice clips were made on the laptop in advance, so there is no cloud API.

Presenter Notes:

- Operation, in order. 1) scripts/stackchan/demo.sh show (wg show): expect the peer, a latest handshake, an endpoint on the phone's network, and transfer counters. 2) demo.sh login (telnet 10.10.0.2): expect the NuttShell prompt. 3) ifconfig inside the session: expect wlan0 and wg0 with 10.10.0.2. 4) stackchan face happy (also neutral, sad, angry, sleepy, surprised), then stackchan say http://10.10.0.1:8000/hello.wav: the clip comes through the same tunnel, the face changes and the mouth moves. Optional: demo.sh outside (tcpdump on eth0, UDP 51820) and demo.sh inside (tcpdump on wg0, TCP 23), then show again. Wait for the board's reaction after each step.
- Which version runs: the robot firmware is the application build (Version 1, FLAT), built by the esp32s3-stackchan stage of the Dockerfile. The kernel driver (Version 2) was run on the same ESP32-S3 in a separate build on 2026-10-12, without the Stack-chan app. Do not say the demo is Version 2.
- **Before the talk:** phone hotspot on (2.4 GHz / maximize compatibility), laptop on it, Docker Desktop running, the Stack-chan joined with `nsh_wifi.py` (the passphrase is not saved on the board), `wg_setup.py` run once. Check `stackchan say` once. If the hotspot was turned off and on, its addresses may change: run `nsh_wifi.py` and `wg_setup.py` again. Rarely (about 1 in 20) the board stops during `say`; power it off and on, then repeat the same steps.
- **Never on screen:** `wg showconf` (prints the private key), the key files in `%USERPROFILE%\stackchan-wg`, the hotspot passphrase.
- **Fallback:** no backup video at the moment. If the live demo stalls, say so and go on to the summary.

31 — Agenda: ⑦ Summary and Reflections (21:07 → 21:17)

Say (one line): Finally, a summary and some reflections.

Pause: point at the highlighted line, one beat.

32 — Summary (21:17 → 21:59)

On slide: Three rows, each a short heading and one sentence: 1 Secure Communication (brought WireGuard to Apache NuttX, secure IP communication with standard peers) / 2 Application to Kernel Driver (evolved from an application into a kernel network driver, tested on real boards and QEMU) / 3 Toward Upstream Contribution (preparing the driver and related fixes for review; a small line: prepared as small, separate changes, nothing is submitted yet).

Say — Essential:

- To sum up. We brought WireGuard to Apache NuttX. A NuttX device can now talk to standard WireGuard peers over an encrypted IP tunnel.
- We developed it from an application into a kernel network driver. We tested it on real boards, and on QEMU in the kernel build modes.
- Next, I am preparing the driver and the related fixes for review by the NuttX community. They are small, separate pieces, and nothing is submitted yet.
- That is the technical story. Now, I'd like to share what I learned.

Presenter Notes:

- If asked 'is it merged?': not yet, and nothing is submitted. These are submission candidates, not a claim that it is mergeable.
- **State of the submission (2026-10-13):** no pull request about this work exists in apache/nuttx or apache/nuttx-apps. The only related issue is the closed GSoC proposal thread (#18548). The branches exist only on the author's forks. Do not say "submitted".

33 — What I Learned (21:59 → 23:23)

On slide: Three cards: from application user to OS contributor / AI helped me code, but understanding was my job (highlighted) / a welcoming open-source community (docs and guides, review, AI-assisted development with people responsible, CI and hackathons help new people join). Bottom line: the hope to keep contributing.

Say — Essential:

- Finally, I'd like to share three things I learned.
- First, I came to NuttX as an application developer. This time, I worked inside the OS: drivers and the kernel.
- Second, and I want to say this clearly: I used AI, Claude, a lot in this project. It made research and coding much faster. But embedded OS internals were new to me. I could not just take the code the AI wrote. I asked questions, I read the NuttX source, and I tested the behavior, until I could explain it myself. AI helped me code, but understanding was my job.
- Third, the community. NuttX has good documentation and contributing guides, and active discussion and review. It is open to AI-assisted development, and people stay responsible for their work. CI and hackathons also help new people join. Open source is not only public code. It grows when new people can join, learn, and contribute.
- I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.

Say — Optional:

- In NuttX, a commit says Assisted-by when AI helped, and AI never adds a Signed-off-by.

Presenter Notes:

- Say the AI part plainly: not as a confession and not as a boast. It was faster, and understanding was still hard and still mine.

34 — Acknowledgements (23:23 → 24:09)

On slide: A plain vertical list, a name and one line each: Alan Carvalho de Assis (encouraged me to submit this talk) / The Apache NuttX Community (discussions, documentation, support) / Sony's NuttX Developers, Past and Present (their contributions and the foundation they built) / The Apache Software Foundation & Community Over Code Organizers (bringing the community together).

Say — Essential:

- Before I finish, I'd like to say thank you.
- First, Alan Carvalho de Assis, who encouraged me to submit this talk to Community Over Code.
- Second, the Apache NuttX community, for the discussions, the documentation, and the support. When I wrote my GSoC proposal, people encouraged me. That is why I kept going after it was not selected.
- Third, the Sony engineers before me, who contributed to NuttX and built the foundation for this work.
- And finally, the Apache Software Foundation, and the organizers and volunteers of Community Over Code, for bringing the community together.

35 — Thank You — Questions? (24:09 → 24:26)

On slide: A QR code of the slides ("Scan to view the slides") with the slides link and the GitHub repository link, both clickable. The same QR code is on the title slide. No video link.

Say — Essential:

- Thank you very much. I am happy to take your questions.
- The slides are on the screen. You can scan the QR code, or use the link. The code and the records are on GitHub.

Say — Optional:

- Questions about the kernel driver, the bugs, or the hardware are all welcome.

Presenter Notes:

- **After:** keep coc-glasgow-qa.md at hand, not a slide. The usual questions (rebase status, #14, IOB exhaustion, checkpatch) are answered there and on no slide.
- **Check before the talk:** the QR code and both links open https://wwlapaki310.github.io/gsoc2026-nuttx-wireguard/docs/presentation/coc-glasgow-reveal/index.html (GitHub Pages must be up to date with the last push).

A1 — Testing: What Ping Misses (appendix, not timed)

On slide: Three cards: same key on every boot? (SPRESENSE, 3 resets, 3 keys) / keys left after down? (sim, all zero) / a leak after hours? (sim, 151 rekeys, 100 down/up, nothing lost).

Say — Essential:

- This slide is about what a working tunnel cannot tell you: three problems that ping would never notice.
- First, keys. NuttX can seed its random device from build-time settings. Then a board makes the same private key every time it starts, and everything still works. So I reset a SPRESENSE three times, and I got three different keys. That is a quick check, not a proof of good randomness.
- Second, I read the device's memory in the simulator. After down, the session keys were all zero. The static key stays on purpose, so that up works again.

- Third, a long test, also in the simulator: one hundred and fifty-one forced rekeys and one hundred down-and-up cycles. No buffer was lost, and memory went back to exactly the start-up number. I am not saying it ran for a week, or that the simulator behaves like the board.

A2 — Design Choice: Trust the Clock? (appendix, not timed)

On slide: The rule (a newer timestamp, even after a reboot), Chosen (real-time clock + the highest value this boot, plus a warning) / Not chosen (save every time, reserve a range in flash, wait), and the measured pair: clock unset → no handshake in 75 s, clock set → 4.1 s.

Say — Essential:

- This is the one place where I had to decide, not test. And then I measured my decision.
- WireGuard's replay protection has a rule: every handshake needs a newer timestamp than the peer has seen, even after a reboot. The device is usually the side that starts the handshake. It is also the side least likely to have a clock with a battery.
- Saving every timestamp wears out the flash. Reserving a range in flash is the right long-term answer, but it needs a write guarantee that I could not verify. And just waiting means a device that ran for a month cannot reconnect for a month.
- So I chose the real-time clock plus the highest value used in this boot. Linux and wireguard-go do the same. And the driver warns once if the clock was never set.
- Then I measured it on the board. I sent the board nothing at all, because a ping from the peer would start the handshake for it. With the clock as it boots, there was no handshake in a seventy-five-second watch. With the clock set, it took four point one seconds. That is why the driver prints a warning.